

## Exercise 4a

**Deadline: 18.12.2025 22:00.**

Ask questions in discord to #ask-your-tutor

In this exercise, you will familiarize yourselves with simulation-based inference, using the noise-free SIR equations from epidemiology as an example.

## Regulations

Please implement your solutions in form of *Jupyter notebooks* (\*.ipynb files), which can mix executable code, figures and text in a single file. They can be created, edited, and executed in the web browser, in the stand-alone app JupyterLab, or in the cloud via Google Colab and similar services.

Create a Jupyter notebook `epidemiology-part-1.ipynb` for your solution and export the notebook to HTML as `epidemiology-part-1.html`. Zip all files into a single archive `ex04a.zip` and upload this file to MaMPF before the given deadline.

Moreover, please set your **Anzeigename/display name** and **Name in Uebungsgruppen/name in tutorials** in MaMPF to your real name, which should be identical to your name in `muesli` and make sure you **join the submission** of your team via the invitation code before the submission deadline. Check out <https://mampf.blog/handing-in-homework-assignments> for instructions.

## 1 Simulation setup

We start with the pure physics-based simulation of an epidemy. We define the compartments “Susceptible” (S), “Infected” (I), and “Recovered” (R), whose dynamics are described by a system of ordinary differential equations (ODEs), known as *SIR equations*:

$$\begin{aligned}\frac{dS}{dt} &= -\lambda \frac{S \cdot I}{N} \\ \frac{dI}{dt} &= \lambda \frac{S \cdot I}{N} - \mu \cdot I \\ \frac{dR}{dt} &= \mu \cdot I\end{aligned}$$

Here,  $N = S + I + R$  is the total size of the population. This simplest form of the SIR model assumes that nobody dies from the infection, and people become permanently immune after recovery. The parameters  $\lambda$  (infection rate),  $\mu$  (recovery rate), and  $I_0$  (initial number of infected people when the disease is first noticed) determine the behavior of the disease, but they cannot be measured directly. Instead, one observes the dynamics of the cases, i.e. the daily number of newly infected and recovered people  $\Delta S(t) = S(t-1) - S(t)$  and  $\Delta R(t) = R(t) - R(t-1)$  (both are taken to be positive by convention, although  $\Delta S$  should conceptionally be negative) for a sequence of days  $t = 1 \dots T$ . Simulation-based inference attempts to estimate  $\lambda$ ,  $\mu$ , and  $I_0$  by calculating backwards from the observations. *Inverse problems* like this are usually not analytically solvable and call for a solution by means of neural networks.

To implement a simulation in form of an SIR model, solve the following tasks:

- Implement a function `simulate_sir_simple(lam, mu, I_0, T)`, that takes the specified parameters and simulates the epidemy using the Euler forward scheme up to day  $T$ . The vector of hidden parameters is therefore  $y = [\lambda, \mu, I_0]$ . The function shall return the sequence of observations

$$x = [(\Delta S(1), \Delta R(1)), \dots, (\Delta S(T), \Delta R(T))]$$

To ensure that  $x$  always has the same dimension, we fix  $T = 100$ . (For now, we want to avoid the complications arising from sequences of varying lengths.) For debugging and visualization, it is also useful to return the sequence of corresponding ODE variables

$$c = [(S(1), I(1), R(1)), \dots, (S(T), I(T), R(T))]$$

but these ground-truth dynamics are not needed for the inverse problem itself, only for testing. Since the simulation in this task is deterministic and noise free, a fixed choice of  $y$  will always lead to the same outcomes  $x$ .

- Run multiple simulations with  $T = 100$  and different  $y$  and visualize the resulting dynamics. Determine suitable ranges of the parameters such that the first 100 days exhibit interesting and diverse dynamics. This is important – when not much happens between day 1 and day  $T$ , there will be insufficient information to solve the inverse problem. Choose parameters such that typical epidemics will not already end by day 30 or unfold only after day 70. Use the results of these experiments to define the simulation prior

$$p^{\text{sim}}(Y) = p^{\text{sim}}(\lambda) \cdot p^{\text{sim}}(\mu) \cdot p^{\text{sim}}(I_0)$$

## 2 Point estimation

We first solve the inverse problem of the SIR equations by an ordinary feed-forward network  $\Phi$  that provides a *point estimate*. That is, it is trained as a regression network to predict a single plausible parameter vector  $\hat{y} = \Phi(x)$  directly from the observations. Note that this network disregards potential uncertainty and ambiguity of the inverse problem. Train this network with the squared loss using a training set generated by the simulation and prior distribution from task 1. Recall that the length of the sequence  $x$ , i.e. the stopping time  $T = 100$ , shall remain fixed for all runs. Evaluate the network's performance by a simulated test set: Visualize the accuracy with suitable diagrams (e.g. plot predicted values vs. ground-truth values) and informative statistics (e.g. the MSE error). Investigate how accuracy depends on the hyperparameters, especially the sizes of the network and training set. Check if there are choices of  $y$  where the inverse problem is more or less difficult (i.e. accuracy is consistently lower or higher than average near these  $y$ 's). Comment on your findings.

## 3 Posterior estimation

Now, we want to use a conditional invertible network (cINN) to predict the full posterior  $p(Y | x)$ . Since the observations are high-dimensional ( $\dim(x) = 200$  for  $T = 100$ ), it is useful to incorporate a summary network  $h(x)$  into the model, which preprocesses  $x$  and represents it by a lower-dimensional feature vector. The network  $h(x)$  should be derived (with suitable modifications) from your best network  $\Phi(x)$  in task 2. For example, remove the last layer from  $\Phi(x)$ , increase the network width, and fine-tune the other hyperparameters. Then implement a conditional coupling flow (RealNVP) to estimate the posterior  $p(Y | h(x))$ , similar to exercise 3. Train the two networks jointly by minimizing the NLL loss over the training set.

Implement the following visualizations for the test set:

- For a given test observation  $x$ , create a plot of the *marginal posterior*  $p(\lambda | h(x))$ , e.g. by forming a histogram from a posterior sample. For comparison, the plot should also show the prior  $p^{\text{sim}}(\lambda)$ , the ground-truth value  $\lambda^*$  of this simulation run, and the point estimate  $\hat{\lambda} = \Phi(x)$  from task 2. Create similar plots for  $\mu$  and  $I_0$ . Repeat this for several simulation runs with different  $y$ 's and check that the ground-truth values are covered by the posteriors (requirement for the posteriors to be correct), the point estimates are located near the center of the posteriors (requirement for the point estimates to be correct), and the posteriors are narrower than the priors (requirement for the data  $x$  to be informative about  $y$ ). If any of these checks turn out poorly, try to improve your networks (it may just be a bug). Comment on your findings.

- Similarly, create 2-dimensional plots for the three possible pairs of variables  $(\lambda, \mu)$ ,  $(\lambda, I_0)$ , and  $(\mu, I_0)$ . For comparison, these plots should show the pairwise posteriors and priors as contour plots or heat maps, along with the ground-truth values and point estimates. Again, check for several test runs if the posteriors cover the true values and are narrower than the priors, and comment on your findings. Especially check if the 2-dimensional posteriors reveal interesting correlations between the hidden parameters.
- Sample  $\hat{y} \sim p(Y | h(x))$  from the full posterior and run the simulation again with these  $\hat{y}$  (“resimulation”) to get a trajectory  $\hat{x}(t)$ . Repeat this with several  $\hat{y}$  from the same posterior, i.e. with the condition  $h(x)$  fixed. Create and plot the 50% confidence region for  $\hat{x}(t)$  from these resimulations and check if the actual  $x$  is covered by this region (should happen about 50% of the time). Repeat this for different  $x$  and comment on your findings.

## 4 Model misspecification check

We now want to verify that test data from the same simulation are recognized as inliers of the training, i.e. as problem instances where the model considers itself competent. To this end, we add a (weighted) MMD term to the loss function of task 3, which pulls the output distribution of the summary network towards a standard normal. That is,  $p(h(x)) \approx \mathcal{N}(0, \mathbb{I})$  should hold after training. (Recall how we used MMD in exercise 2 to pull the latent distribution of a ResNet toward a standard normal – do it likewise here.)

- Repeat the training of task 3 with the modified loss. Check if the new constraint on  $h(x)$  impacts the accuracy of  $p(Y | h(x))$  (this would be undesirable) and comment on your findings.
- Check with a statistical test if test observations  $x$  are recognized as inliers in the summary space  $h(x)$ : If  $p(h(x)) = \mathcal{N}(0, \mathbb{I})$  holds, the squared norms  $\|h(x)\|_2^2$  follow a  $\chi^2$ -distribution with  $\dim(h(x))$  degrees of freedom. Thus, you can get a  $p$ -value by the following Python code

```
import numpy as np
from scipy.stats import chi2

...
s = h(x)                # compute features for a single x ...
s2 = np.sum(s**2)        # and their squared norm
df = s.size              # calculate degrees of freedom
p_value = chi2.sf(s2, df=df) # chi-square test
```

The function `chi2.sf()` evaluates the *survival function* of the  $\chi^2$ -distribution, i.e. the probability  $\Pr[\|u\|_2^2 \geq s2]$  when  $u$  is drawn from a standard normal. If this probability (i.e. the  $p$ -value) is very low, the observed  $s2$  is likely an outlier. Then the model should declare itself incompetent and refuse to process the underlying observations  $x$  any further. Since our test data are created with the same simulation as the training data, this should rarely happen.

- You get a more powerful validation when you apply a Kolmogorov-Smirnov test to the entire test set at once (instead of doing the  $\chi^2$ -test for one instance at a time):

```
import numpy as np
from scipy.stats import kstest, chi2

...
S = h(X)                # compute features for entire test set X ...
S2 = np.sum(S**2, axis=1) # and their squared norm
df = S.shape[1]          # calculate degrees of freedom
stat, p_value = kstest(S2, chi2(df).cdf) # Kolmogorov-Smirnov test
```

The test returns a low  $p$ -value if the difference between the empirical CDF (cumulative distribution function) of the vector  $S2$  and the true CDF of the  $\chi^2$ -distribution is larger than would be expected due to sampling noise. When the test data are from the correct simulation, this shouldn't happen either.